GEBZE TEKNİK ÜNİVERSİTESİ

ELEKTRONİK MÜHENDİSLİĞİ

ELM235

LOJİK DEVRE TASARIM LABORATUVARI

LAB 3 Deney Raporu

Lojik Devreler ve Tasarım Laboratuvarı Dersine Hazırlık

Hazırlayanlar
1) 200102002059 – Rümeysa Çiftçi
2) 210102002808 – Fouad Aladhami

1 İçindekiler

2 Giriş 3
3 Problemler 3
3.1 Problem 1- 2N x 1 Çoğullayıcı (MUX) Tasarımı 3
3.1.1 2x1 MUX Devresinin Sadece NAND Kapısı Kullanılarak Kodunun Yazılması 5
3.1.2 Test Bench Kodunun Yazılması ve Simülasyonu 6
4 Problem 2- 2N x 1 Çoğullayıcı (MUX) ile Temel Lojik Kapıların ve Büyük Çoğullayıcıların Tasarımı 7
4.1.1 Modül Kodunun Yazılması (Tüm Şıklar İçin) 12
4.1.2 TestBench Kodunun Yazılması 16
5 Problem 3 – 2 x 4 Kod Çözücü (Decoder) Tasarımı 19
5.1 Sistemin Doğruluk Tablosu ve Modül Kodunun Yazılması 19
5.2 Sistemin Test Bench Kodunun Yazılması ve Simülasyon 21
6 Problem 4 – 4 x 16 Kod Çözücü (Decoder) Hiyerarşik Tasarımı 22
6.1 2x4 Decoder ve NAND Kapısı Kullanarak 3x8 ile 4x16 Decoder Tasarımı 22
6.2 TestBench Kodunun Yazılması ve Simülasyon 26
7 Problem 5 – Çoğullayıcı (MUX) ve Çözücüler (DECODER) ile Birleşik Devre Tasarımı 28
7.1 F1 ve F2 Fonksiyonlarının Donanım Tasarım Dilleri Kullanılarak Tasarımı 28
7.2 TestBench Kodu ve Simülasyon 37
8 Sonuç ve Yorumlar 39

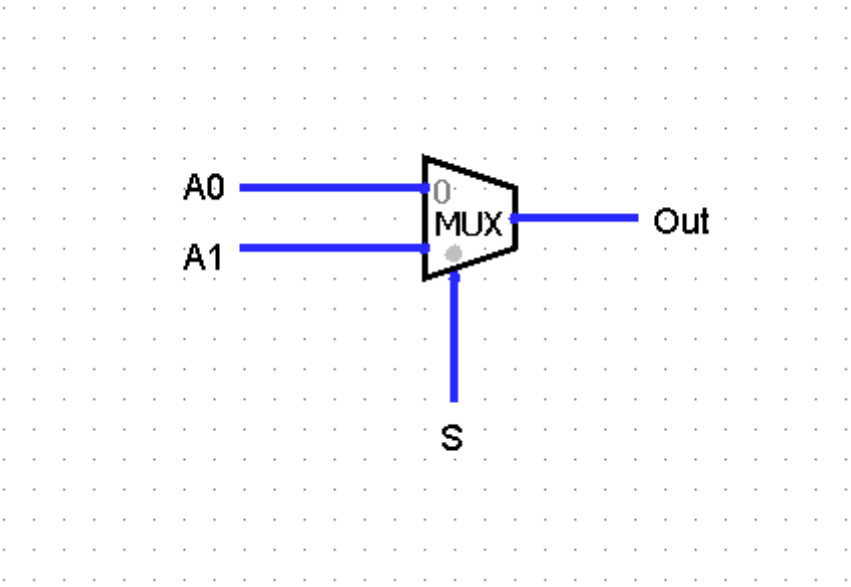
2 Giriş

Bu projede çeşitli amaçlar hedeflenmiştir:

- Çoğullayıcı (MUX) ve Kod Çözücü (Decoder) ‘ın yapısını ve çalışma mantığını anlamak.
- MUX ile temel lojik kapıların tasarımı.
- Hiyerarşik tasarımın nasıl yapıldığının öğrenilmesi ve System Verilogla gerçekleştirilmesi.
- MUX ile özel fonksiyonların gerçekleşmesi .
- Devreleri sentezlenmesi ve yapısının incelenmesi.

3 Problemler

3.1 Problem 1- 2N x 1 Çoğullayıcı (MUX) Tasarımı



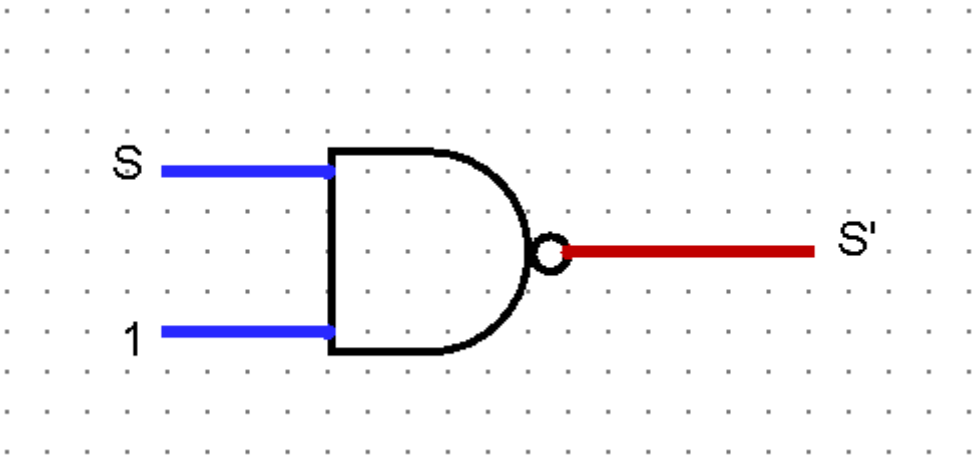
Şekil 1

S0	Y
0	A0
1	A1

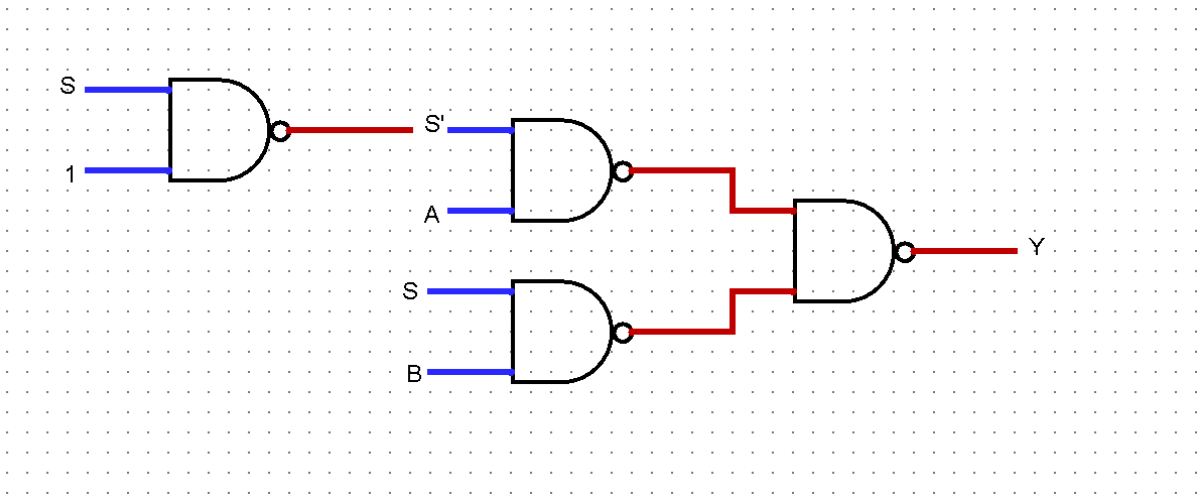
Şekil 2

$$De\ Morgan\ Kuralı:\ (m+n)\ =\ (m'n')'$$

$$Y\ =\ S0'\ A0\ +\ S0\ A1\ =\ ((\ S0'\ A0\ )\ '\ (S0\ B)'\ )\ =\ (S0'\ nand\ A0\ )\ nand\ (S0\ nand\ B)$$



Şekil 3: S' elde edilmesi(NAND kullanarak sinyalin tersinin alınması)



Şekil 4: NAND Kapılarıyla 2x1 MUX tasarımı

3.1.1 2x1 MUX Devresinin Sadece NAND Kapısı Kullanılarak Kodunun Yazılması

```
/* lab3_g5_p1.sv */

`timescale 1ns/1ns

module lab3_g5_p1(

    //this is a 2x1 MUX

    input logic [1:0] A,

    input  logic S,

    output logic Y

    );

    // Y = ((S nand 1) nand A[0]) nand (S nand A[1])


    logic nand1, nand2, nand3;

    NAND n1(S, 1'b1, nand1);

    NAND n2(nand1, A[0], nand2);

    NAND n3(S, A[1], nand3);

    NAND n4(nand2, nand3, Y);

    //assign nand1 = ~(S & 1'b1);

    //assign nand2 = ~(nand1 & A[0]);

    //assign nand3 = ~(S & A[1]);

    //assign Y = ~(nand2 & nand3);


endmodule

module NAND(a, b, y);


input logic a, b;

output logic y;


    assign y = ~(a & b);

endmodule
```

3.1.2 Test Bench Kodunun Yazılması ve Simülasyonu

```
/* tb_lab3_g5_p1.sv */

`timescale 1ns/1ns

module tb_lab3_g5_p1();

    logic [1:0] A;

    logic S,Y;

    lab3_g5_p1 dut1(A,S,Y);//device under test

    initial begin

        A[1]=0;A[0]=0;S=0 ;#10;

        if ( Y!=0) $display("For A=%b S=%d, yexpected = 0 but Y=%d FAIL",A,S,Y);

        A[1]=0;A[0]=0;S=1 ;#10;

        if ( Y!=0) $display("For A=%b S=%d, yexpected = 0 but Y=%d FAIL",A,S,Y);

        A[1]=0;A[0]=1;S=0 ;#10;

        if ( Y!=1) $display("For A=%b S=%d, yexpected = 1 but Y=%d FAIL",A,S,Y);

        A[1]=0;A[0]=1;S=1 ;#10;

        if ( Y!=0) $display("For A=%b S=%d, yexpected = 0 but Y=%d FAIL",A,S,Y);

        A[1]=1;A[0]=0;S=0 ;#10;

        if ( Y!=0) $display("For A=%b S=%d, yexpected = 0 but Y=%d FAIL",A,S,Y);

        A[1]=1;A[0]=0;S=1 ;#10;

        if ( Y!=1) $display("For A=%b S=%d, yexpected = 1 but Y=%d FAIL",A,S,Y);

        A[1]=1;A[0]=1;S=0 ;#10;

        if ( Y!=1) $display("For A=%b S=%d, yexpected = 1 but Y=%d FAIL",A,S,Y);

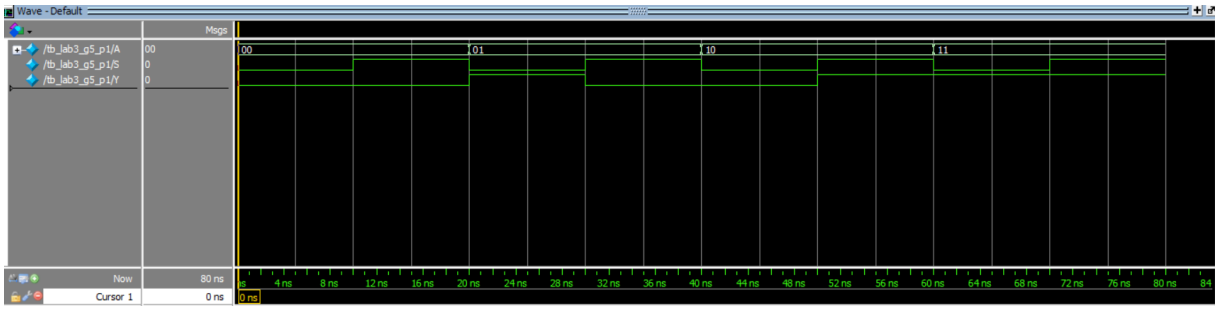
        A[1]=1;A[0]=1;S=1 ;#10;

        if ( Y!=1) $display("For A=%b S=%d, yexpected = 1 but Y=%d FAIL",A,S,Y);

        $stop;


    end

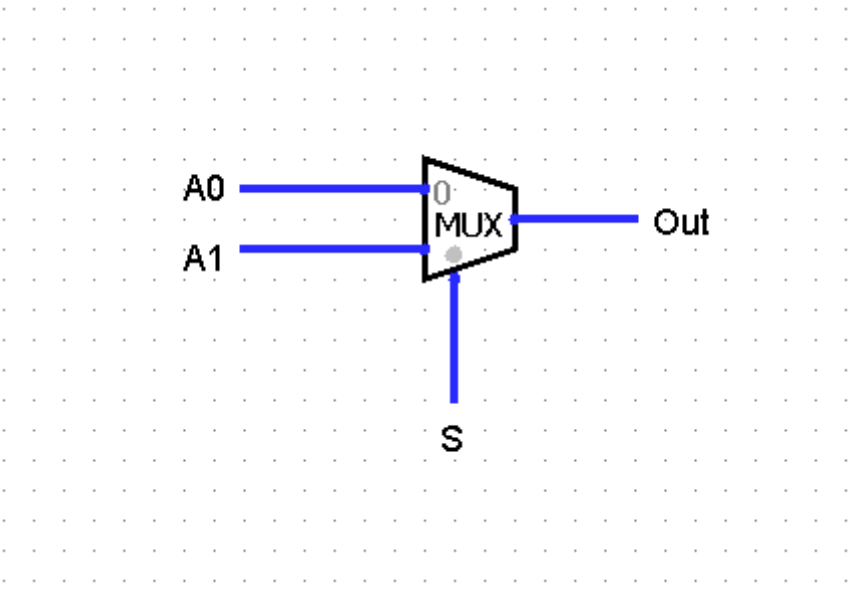
endmodule
```



Şekil 5: Kodun Simülasyonu ve Waveform

4 Problem 2- 2N x 1 Çoğullayıcı (MUX) ile Temel Lojik Kapıların ve Büyük Çoğullayıcıların Tasarımı

$$Y = S' A0 + S A1$$

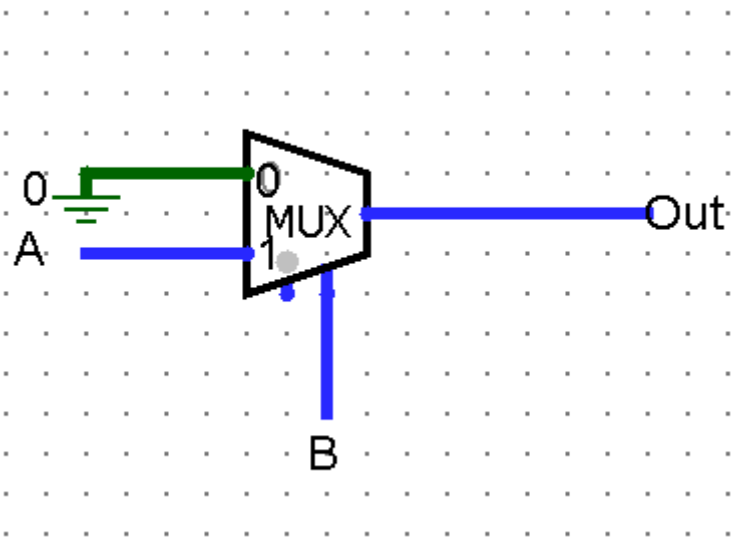


Şekil 1

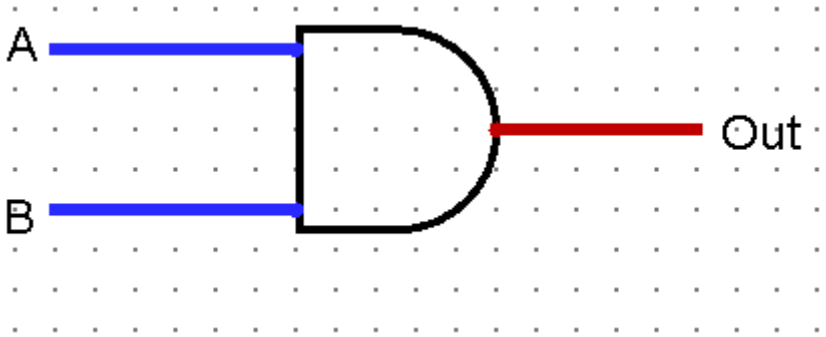
Eğer A_0'ı 0 yani toprak seçersek AND kapısı olur.

Yeni fonksiyon Y = S A_1

Eğer A, B girişli bir AND kapısı tasarlanacaksa A_0 = 0, A_1 = A ve S = B şeklinde tasarlanabilir.



Şekil 2: MUX kullanarak AND kapısı tasarlanması



Şekil 3: AND kapısı gösterimi

OR kapısını ele alalım:

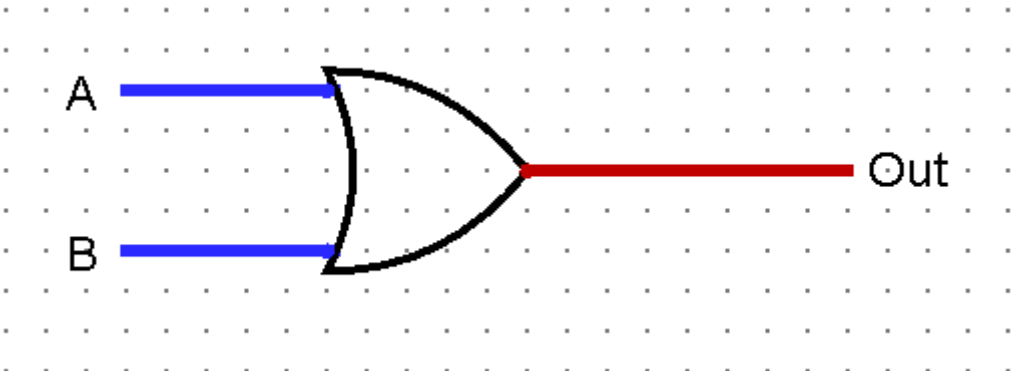
$$A1 = 1\ olsun.$$

$$Y = S'\,A0 + S\ olur.$$

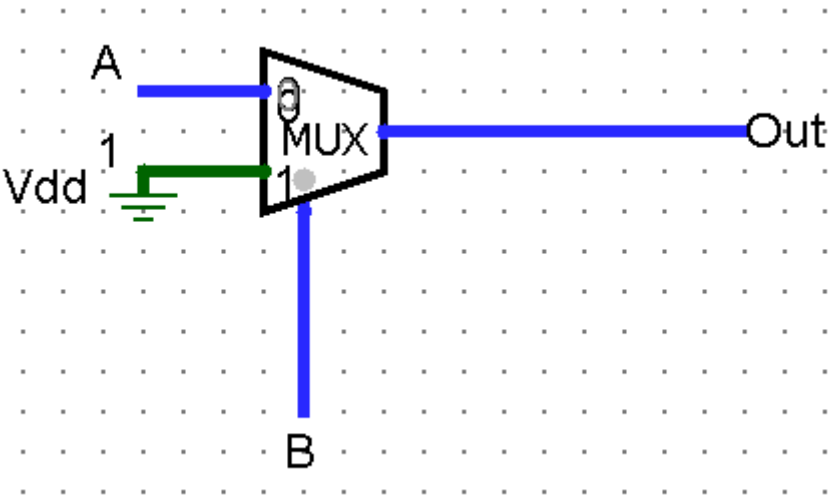
$$Y = S\,(1 + A0) + S'\,A0 = S + S\,A0 + S'\,A0$$

$$Y = A0\,(S + S') + S$$

$$Y = A0 + S = A + B$$



Şekil 4



Şekil 5

NAND kapısını ele alalım:

A,B girişli ise:

$$Y = A'B' - A' + B' \; olmalı.$$

$$Y = S' \, A0 + S \, A1$$

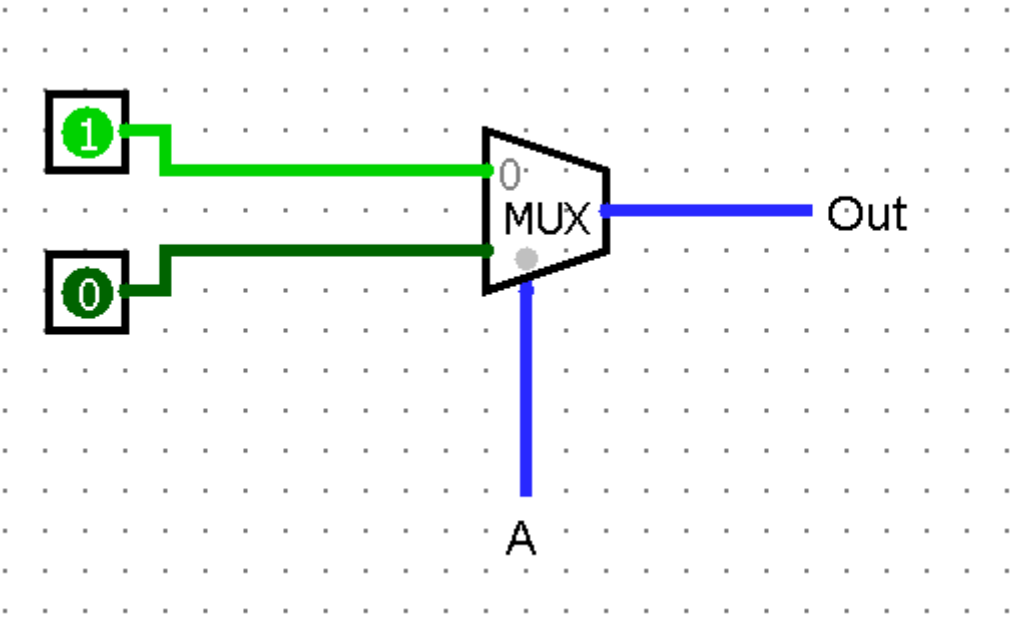
$$A0 = 1 \; olursa,$$

$$Y = S' + S \, A1 \; olur.$$

$$Y = S' + A1$$

$$A1 = A' \; ve \; S = B \; olursa \; NAND \; kapısı \; elde \; edilir.$$

A’ sinyali için inverter’a ihtiyaç duyulur. Bunun tasarımı ise şu şekildedir:

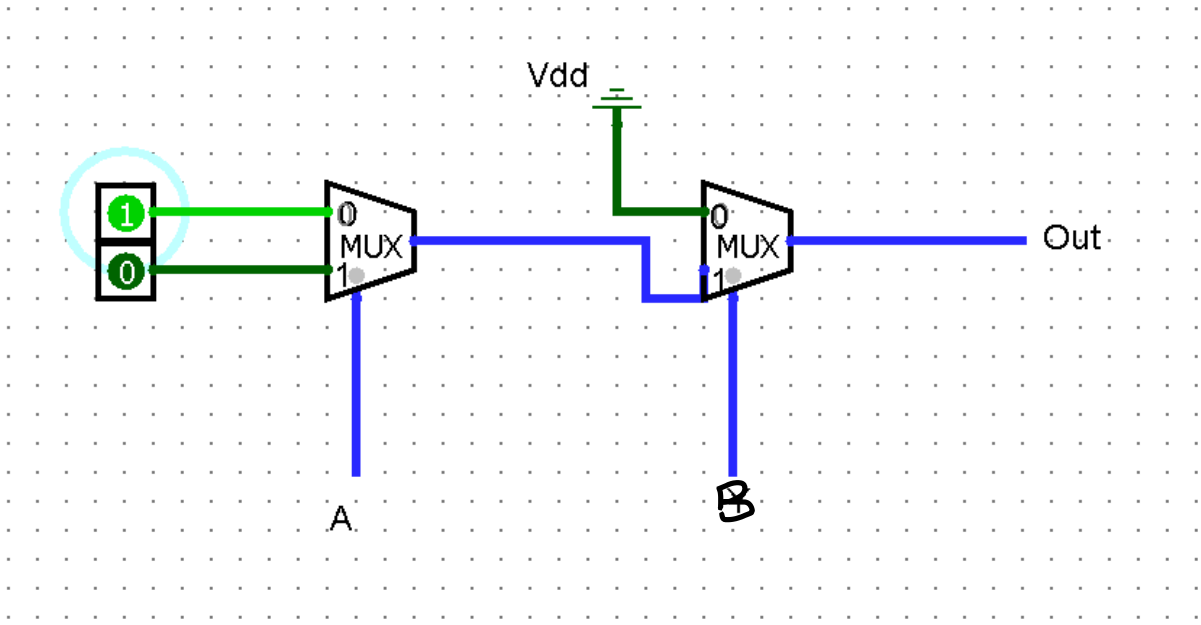


Şekil 6: A' için NOT kapısı tasarımı



Şekil 7: NOT kapısı gösterimi

Sonuç olarak NAND kapısı şu şekilde tasarlanır:



Şekil 8

NOR kapısını ele alalım.

A ve B girişler olsun.

$$Y = (A + B)' = A' B'$$

2x1 Multiplexer versiyonu için:

$$Y = S0' A0 + S0 A1$$

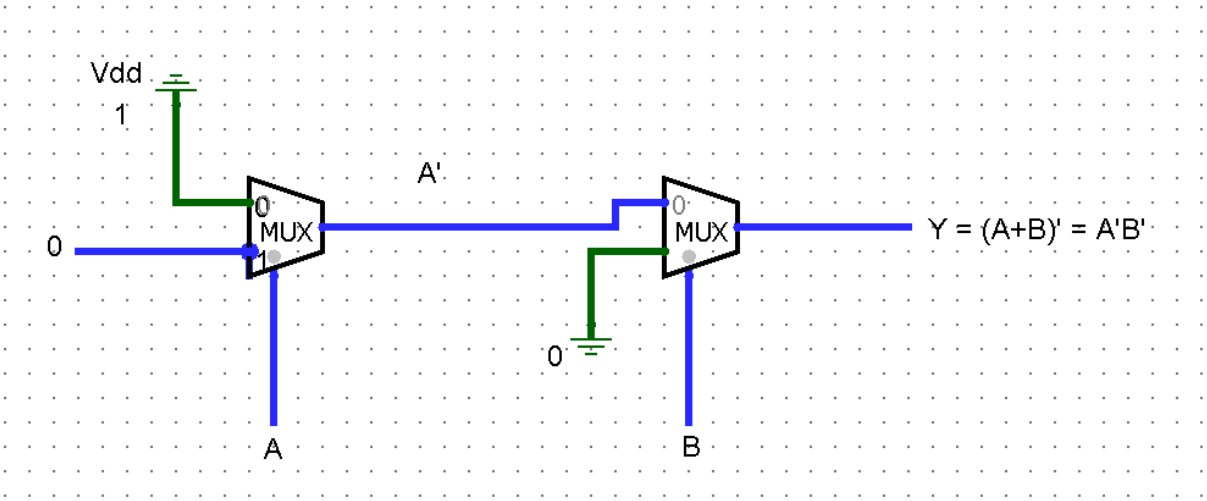
A1'i 0 seçtiğimizde:

$$Y = S0' A0 \; olur.$$

$$A0 = A \; ve \; S0 = B \; olursa$$

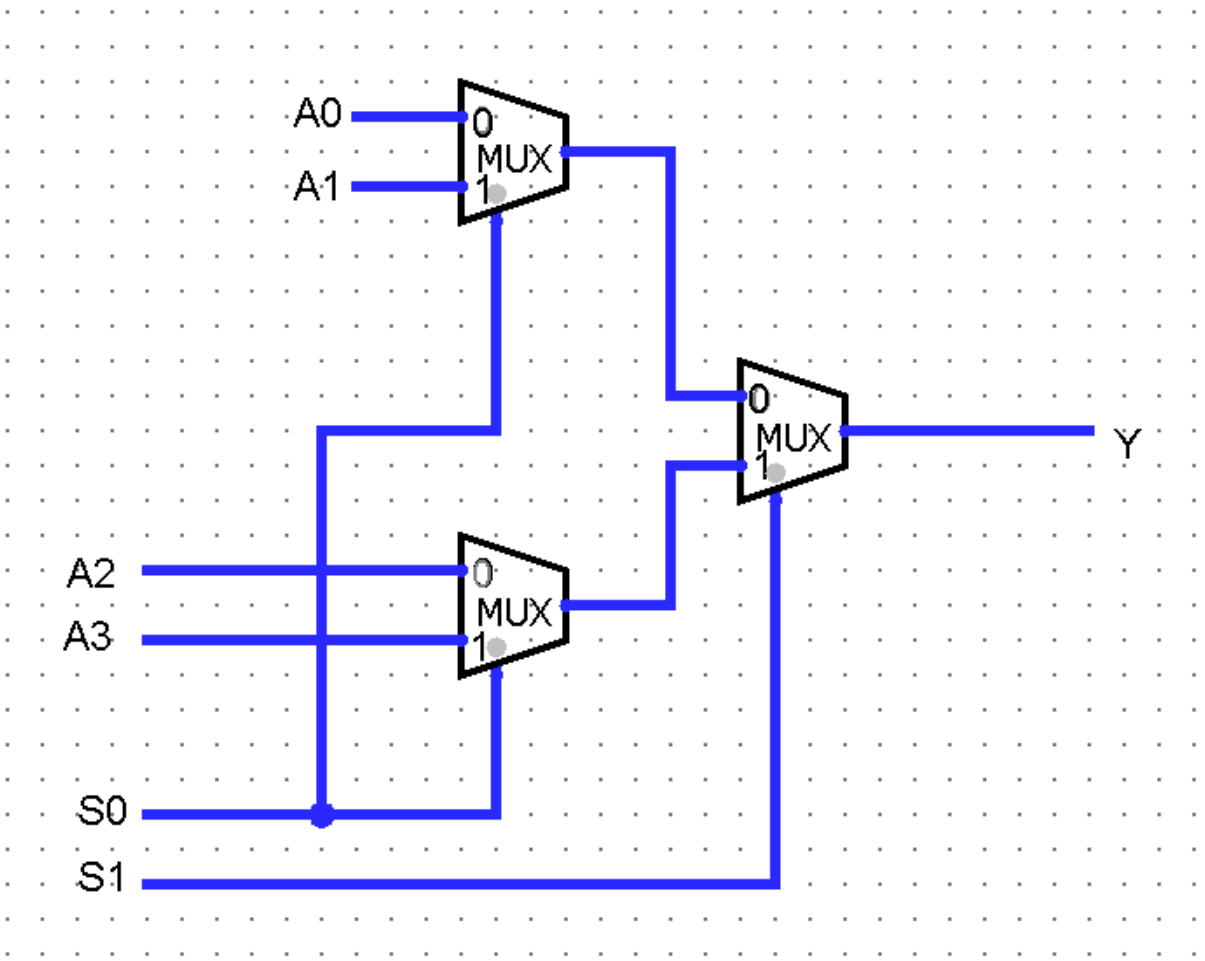
istediğimiz tasarıma ulaşmış oluruz.

A için önceden tasarladığımız inverter kapısının kullanıldığını varsayalım.



Şekil 9

2x1 Multiplexer ile 4x1 Multiplexer tasarımı aşağıdaki gibidir:



Şekil 10

4.1.1 Modül Kodunun Yazılması (Tüm Şıklar İçin)

```
/* lab3_g5_p2.sv */

module MUX16x1(input logic [15:0] A , input logic [3:0] S, output logic Y);

    //A[0] = A; A[1] = A; A[2] = A; A[3] = A; A[4] = A; A[5] = A; A[6] = A; A[7] = A; A[8] = A;
A[9] = A; A[10] = A; A[11] = A; A[12] = A; A[13] = A; A[14] = A; A[15] = A; S = S;

    logic [15:0] Aarray;

    logic y1, y2;

    MUX8x1 mux1(A[7:0], S[2:0], y2);

    MUX8x1 mux2(A[15:8], S[2:0], y1);

    lab3_g5_p1 mux3({y1,y2}, S[3], Y);

endmodule

module MUX8x1(input logic [7:0] A , input logic [2:0] S, output logic Y);

    //A[0] = A; A[1] = A; A[2] = A; A[3] = A; A[4] = A; A[5] = A; A[6] = A; A[7] = A; S = S;

    logic [7:0] Aarray;

    logic y1, y2;

    MUX4x1 mux1(A[3:0], S[1:0], y2);

    MUX4x1 mux2(A[7:4], S[1:0], y1);

    lab3_g5_p1 mux3({y1,y2}, S[2], Y);

endmodule

module MUX4x1(input logic [3:0] A, input logic [1:0] S, output logic Y);

    //A[0] = A; A[1] = A; A[2] = A; A[3] = A; S = S;

    logic [3:0] Aarray;

    logic y1, y2;

    lab3_g5_p1 mux1(A[1:0], S[0], y2);

    lab3_g5_p1 mux2(A[3:2], S[0], y1);

    lab3_g5_p1 mux3({y1,y2}, S[1], Y);

endmodule
```

Kodun devamı şu şekildedir:

```
module NOR_MUX(input logic A,B, output logic Y);

    //A[0] = A; A[1] = A; S = B;

    logic [1:0] Aarray;

    NOT_MUX not1(A, Anot);

    always_comb begin

        Aarray[0] = Anot;

        Aarray[1] = 0;

    end

    lab3_g5_p1 mux(Aarray, B, Y);

endmodule

module NAND_MUX(input logic A,B, output logic Y);

    //A[0] = A'; A[1] = 1; S = B;

    logic Anot;

    logic [1:0] Aarray;

    NOT_MUX not1(A, Anot);

    always_comb begin

        Aarray[0] = 1;

        Aarray[1] = Anot;

    end

    lab3_g5_p1 mux(Aarray, B, Y);

endmodule

module NOT_MUX(input logic A, output logic Y);

    //A[0] = A; A[1] = A; S = B;

    logic [1:0] Aarray;

    always_comb begin

        Aarray[0] = 1;

        Aarray[1] = 0;

    end

    lab3_g5_p1 mux(Aarray, A, Y);

endmodule
```

Kodun devamı şu şekildedir:

```
module AND_MUX(input logic A,B, output logic Y);

    //A[0] = 0; A[1] = A; S = B;

    logic [1:0] Aarray;

    always_comb begin

        Aarray[0] = 0;

        Aarray[1] = A;

    end

    lab3_g5_p1 mux(Aarray, B, Y);

endmodule

module OR_MUX(input logic A,B, output logic Y);

    //A[0] = A; A[1] = 1; S = B;

    logic [1:0] Aarray;

    always_comb begin

        Aarray[0] = A;

        Aarray[1] = 1;

    end

    lab3_g5_p1 lab3_g5_p1(Aarray, B, Y);

endmodule

module lab3_g5_p1(

    //this is a 2x1 MUX

    input logic [1:0] A,

    input  logic S,

    output logic Y

    );

    // Y = ((S nand 1) nand A[0]) nand (S nand A[1])

    logic nand1, nand2, nand3;

    NAND n1(S, 1'b1, nand1);

    NAND n2(nand1, A[0], nand2);

    NAND n3(S, A[1], nand3);

    NAND n4(nand2, nand3, Y);

    //assign nand1 = ~(S & 1'b1);

    //assign nand2 = ~(nand1 & A[0]);

    //assign nand3 = ~(S & A[1]);

    //assign Y = ~(nand2 & nand3);


endmodule
```

Kodun devamı şu şekildedir:

```
module NAND(a, b, y);


    input logic a, b;

    output logic y;


    assign y = ~(a & b);

endmodule
```

4.1.2 TestBench Kodunun Yazılması

```
/* tb_lab3_g5_p1.sv */

`timescale 1ns/1ns

module tb_lab3_g5_p2();


    logic A,B,Yand,Yor,YnotA,Ynand,Ynor, Y4bitmux,Y8bitmux, Y16bitmux;

    logic [15:0] Aarray;

    logic [4:0] S;

    AND_MUX dut1(A,B,Yand);//device under test 1

    OR_MUX dut2(A,B,Yor);//device under test 2

    NOT_MUX dut3(A,YnotA);//device under test 3

    NAND_MUX dut4(A,B,Ynand);//device under test 4

    NOR_MUX dut5(A,B,Ynor);//device under test 5

    MUX4x1 dut6(Aarray[3:0], S[1:0],Y4bitmux);//device under test 6

    MUX8x1 dut7(Aarray[7:0], S[2:0],Y8bitmux);//device under test 7

    MUX16x1 dut8(Aarray[15:0], S[3:0],Y16bitmux);//device under test 8
```

Kodun devamı şu şekildedir:

```
initial begin


        A=0;B=0;#10;
        if ( Yand!=0) $display("For A=%b B=%b yexpected = 0 but Yand=%d FAIL",A,B,Yand);
        if ( Yor!=0) $display("For A=%b B=%b yexpected = 0 but Yor=%d FAIL",A,B,Yor);
        if ( YnotA!=1) $display("For A=%b yexpected = 1 but YnotA=%d FAIL",A,YnotA);
        if (Ynand!=1) $display("For A=%b B=%b yexpected = 1 but Ynand=%d FAIL",A,B,Ynand);
        if (Ynor!=1) $display("For A=%b B=%b yexpected = 1 but Ynor=%d FAIL",A,B,Ynor);
        A=0;B=1;#10;
        if ( Yand!=0) $display("For A=%b B=%b yexpected = 0 but Yand=%d FAIL",A,B,Yand);
        if ( Yor!=1) $display("For A=%b B=%b yexpected = 1 but Yor=%d FAIL",A,B,Yor);
        if ( YnotA!=1) $display("For A=%b yexpected = 1 but YnotA=%d FAIL",A,YnotA);
        if (Ynand!=1) $display("For A=%b B=%b yexpected = 1 but Ynand=%d FAIL",A,B,Ynand);
        if (Ynor!=0) $display("For A=%b B=%b yexpected = 0 but Ynor=%d FAIL",A,B,Ynor);
        A=1;B=0;#10;
        if ( Yand!=0) $display("For A=%b B=%b yexpected = 0 but Yand=%d FAIL",A,B,Yand);
        if ( Yor!=1) $display("For A=%b B=%b yexpected = 1 but Yor=%d FAIL",A,B,Yor);
        if ( YnotA!=0) $display("For A=%b yexpected = 0 but YnotA=%d FAIL",A,YnotA);
        if (Ynand!=1) $display("For A=%b B=%b yexpected = 1 but Ynand=%d FAIL",A,B,Ynand);
        if (Ynor!=0) $display("For A=%b B=%b yexpected = 0 but Ynor=%d FAIL",A,B,Ynor);
        A=1;B=1;#10;
        if ( Yand!=1) $display("For A=%b B=%b yexpected = 1 but Yand=%d FAIL",A,B,Yand);
        if ( Yor!=1) $display("For A=%b B=%b yexpected = 1 but Yor=%d FAIL",A,B,Yor);
        if ( YnotA!=0) $display("For A=%b yexpected = 0 but YnotA=%d FAIL",A,YnotA);
        if (Ynand!=0) $display("For A=%b B=%b yexpected = 0 but Ynand=%d FAIL",A,B,Ynand);
        if (Ynor!=0) $display("For A=%b B=%b yexpected = 0 but Ynor=%d FAIL",A,B,Ynor);


        $stop;
    end
```

Kodun devamı şu şekildedir:

```
integer i,j;

    initial begin

        for (i = 0; i < 16; i = i + 1) begin

            for (j = 0; j < 65536 ; j = j + 1) begin

                S = i;

                Aarray = j;

                #10;//burada 10ns bekliyoruz

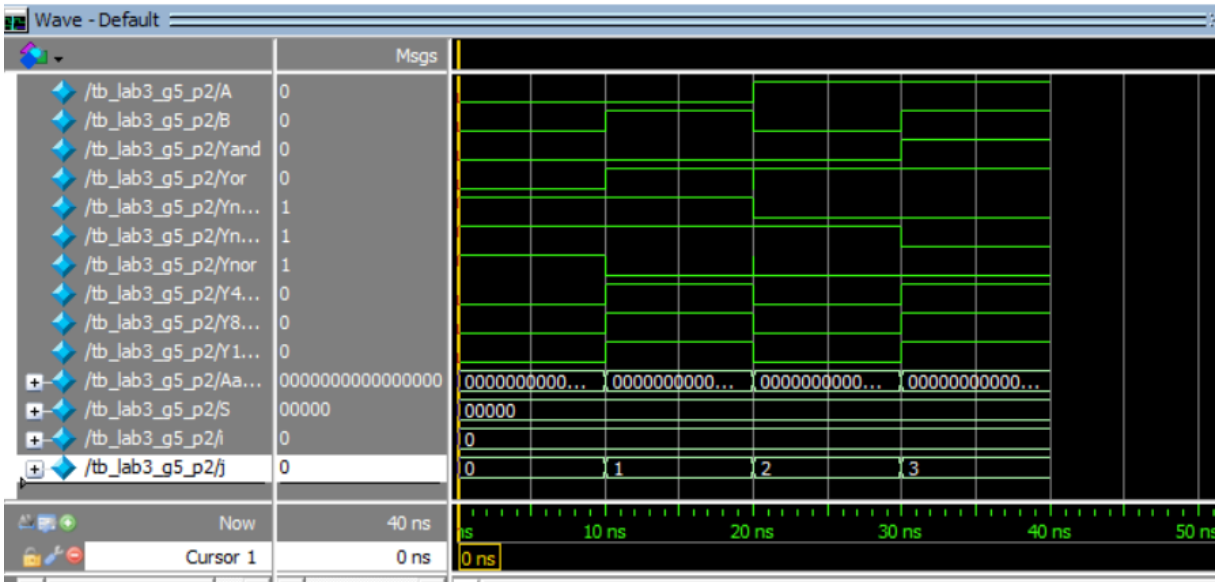
            end

        end

        $stop

    end


endmodule
```



Şekil 11: Kodun Simülasyonu ve Waveform

5 Problem 3 – 2 x 4 Kod Çözücü (Decoder) Tasarımı

5.1 Sistemin Doğruluk Tablosu ve Modül Kodunun Yazılması

2X4 decoder tasarımı için doğruluk tablosu aşağıdaki gibidir:

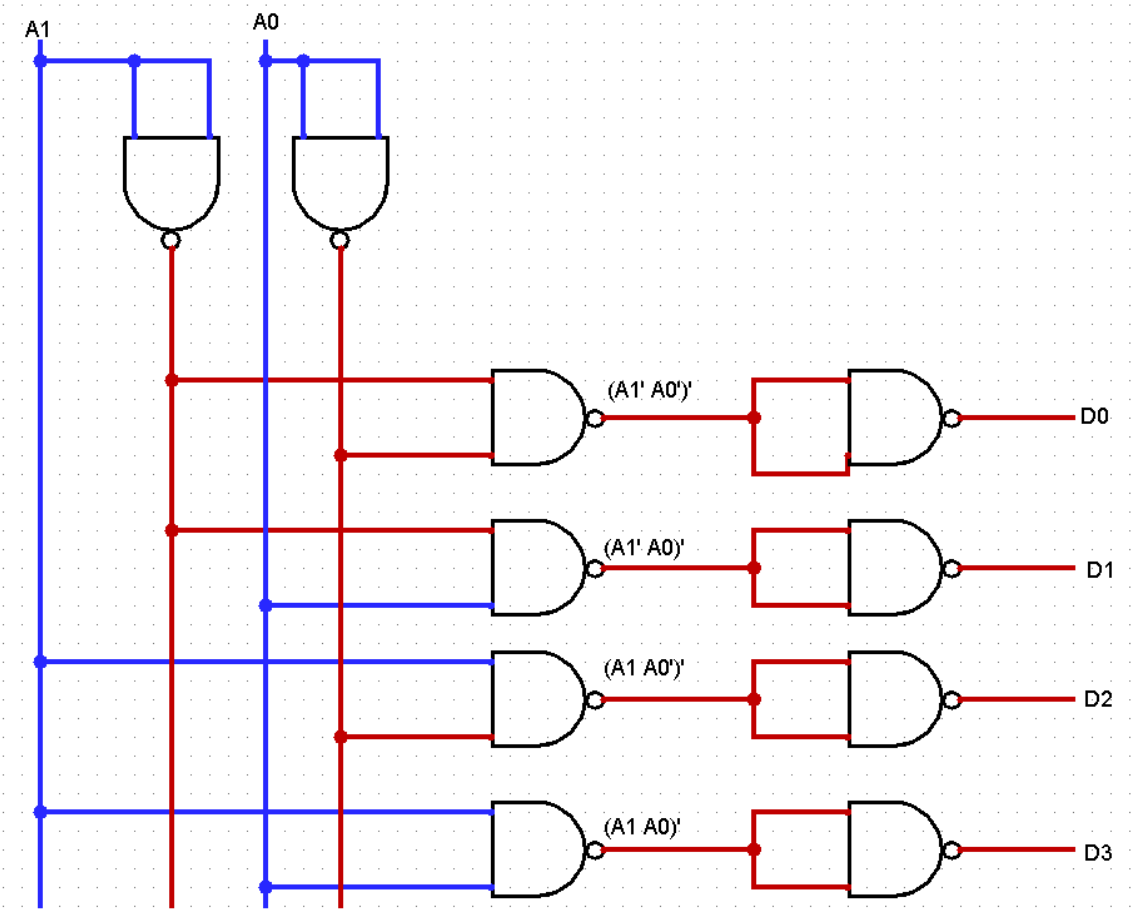
A1	A0	D3	D2	D1	D0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

$$D0 = A1' A0' = ((A1' A0')')'$$

$$D1 = A1' A0 = ((A1' A0)')'$$

$$D2 = A1 A0' = ((A1 A0')')'$$

$$D3 = A1 A0 = ((A1 A0)')'$$



Şekil 12

```
/* tb_lab3_g5_p3.sv */

`timescale 1ns/1ns

module lab3_g5_p3(

    //this is a 2x4 decoder

    input logic [1:0] A,

    input logic E,

    output logic [3:0] D

    );

    /*

    A1not = A[1] nand A[1]

    A0not = A[0] nand A[0]

    D[0] = (A1not nand A0not) nand (A1not nand A0not)

    D[1] = (A1not nand A0) nand (A1not nand A0)

    D[2] = (A1 nand A0not) nand (A1 nand A0not)

    D[3] = (A1 nand A0) nand (A1 nand A0)

    */

    logic A1not, A0not , D0_not, D1_not, D2_not, D3_not;

    logic [3:0] buffer;

    NAND A1not_nand(A[1], A[1], A1not);

    NAND A0not_nand(A[0], A[0], A0not);

    NAND D0_not_nand(A1not, A0not, D0_not);

    NAND D1_not_nand(A1not, A[0], D1_not);

    NAND D2_not_nand(A[1], A0not, D2_not);

    NAND D3_not_nand(A[1], A[0], D3_not);

    NAND D0_nand(D0_not, D0_not, buffer[0]);

    NAND D1_nand(D1_not, D1_not, buffer[1]);

    NAND D2_nand(D2_not, D2_not, buffer[2]);

    NAND D3_nand(D3_not, D3_not, buffer[3]);

    assign D = E ? buffer : 4'b0000;

endmodule

module NAND(a, b, y);

input logic a, b;

output logic y;

    assign y = ~(a & b);

endmodule
```

Sistemin modül kodu yukarıda verilmiştir.

5.2 Sistemin Test Bench Kodunun Yazılması ve Simülasyon

```
/* tb_lab3_g5_p3.sv */

`timescale 1ns/1ns

module tb_lab3_g5_p3();

    logic [1:0] A;

    logic [3:0] D2x4;

    logic E;

    lab3_g5_p3 dut1(A,E,D2x4);//device under test 1

    initial begin

        A[1]=0;A[0]=0;E= 0 ;#10;

        if (D2x4!=4'b0000) $display("D2x4=0000 is expected but D2x4=%b is observed",D2x4);

        A[1]=0;A[0]=1;#10;

        if (D2x4!=4'b0000) $display("D2x4=0000 is expected but D2x4=%b is observed",D2x4);

        A[1]=1;A[0]=0;#10;

        if (D2x4!=4'b0000) $display("D2x4=0000 is expected but D2x4=%b is observed",D2x4);

        A[1]=1;A[0]=1;#10;

        if (D2x4!=4'b0000) $display("D2x4=0000 is expected but D2x4=%b is observed",D2x4);

        E = 1;

        A[1]=0;A[0]=0;#10;

        if (D2x4!=4'b0001) $display("D2x4=0001 is expected but D2x4=%b is observed",D2x4);

        A[1]=0;A[0]=1;#10;

        if (D2x4!=4'b0010) $display("D2x4=0010 is expected but D2x4=%b is observed",D2x4);

        A[1]=1;A[0]=0;#10;

        if (D2x4!=4'b0100) $display("D2x4=0100 is expected but D2x4=%b is observed",D2x4);

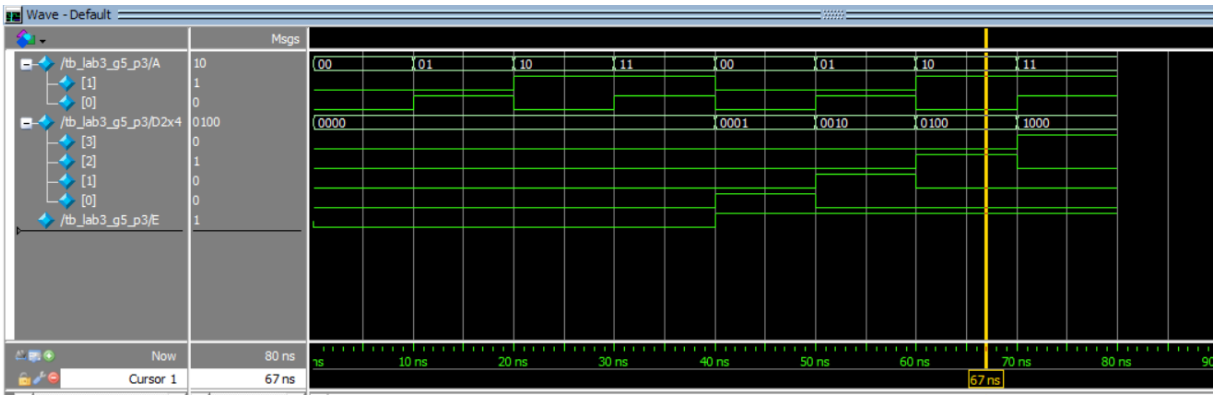
        A[1]=1;A[0]=1;#10;

        if (D2x4!=4'b1000) $display("D2x4=1000 is expected but D2x4=%b is observed",D2x4);


        /$stop;

    end

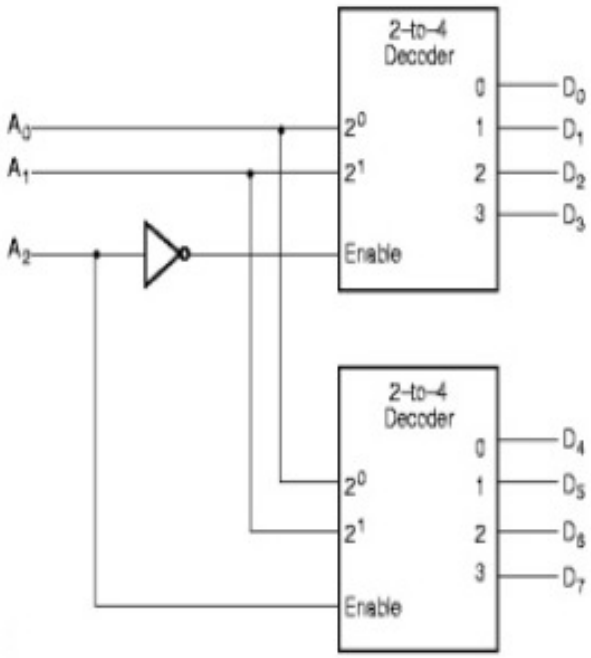
endmodule
```



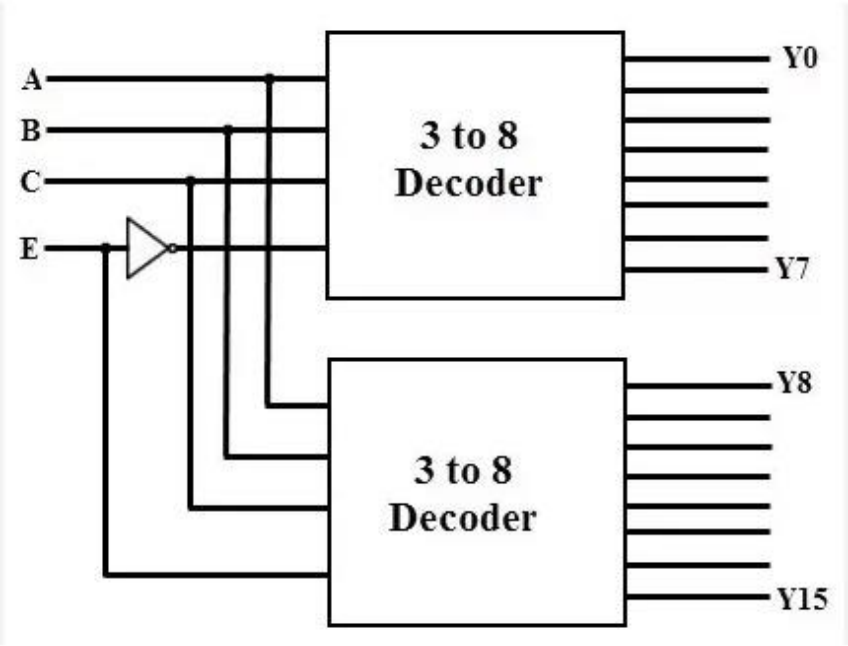
Şekil 13: Kodun Simülasyonu ve Waveform

6 Problem 4 – 4 x 16 Kod Çözücü (Decoder) Hiyerarşik Tasarımı

6.1 2x4 Decoder ve NAND Kapısı Kullanarak 3x8 ile 4x16 Decoder Tasarımı



Şekil 14: 2x4 Decoder Kullanılarak 3x8 Decoder Tasarımı



Şekil 15: 3x8 Decoder Kullanılarak 4x16 Decoder Tasarımı

Devrenin modül kodu aşağıdaki gibi yazılmıştır:

```
/* lab3_g5_p4.sv */

`timescale 1ns/1ns

module lab3_g5_p4(

    //this is a 4x16 decoder

    input logic [3:0] A,

    input logic E,

    output logic [15:0] D);

    logic A3not;logic [15:0] buffer;


    NAND A3not_nand(A[3], A[3], A3not);

    decoder3x8 decoder1(A[2:0], A3not, buffer[7:0]);

    decoder3x8 decoder2(A[2:0], A[3], buffer[15:8]);

    assign D = E ? buffer : 16'b0000000000000000;

endmodule

module decoder3x8(

    //this is a 3x8 decoder

    input logic [2:0] A,

    input logic E,

    output logic [7:0] D);

    logic A2not;logic [7:0] buffer;

    /*

    A1not = A[2] nand A[2]


    */

    NAND A2not_nand(A[2], A[2], A2not);

    lab3_g5_p3 decoder1(A[1:0], A2not, buffer[3:0]);

    lab3_g5_p3 decoder2(A[1:0], A[2], buffer[7:4]);

    assign D = E ? buffer : 8'b00000000;


endmodule
```

Kodun devamı aşağıdaki gibidir:

```
module lab3_g5_p3(

    //this is a 2x4 decoder

    input logic [1:0] A,

    input logic E,

    output logic [3:0] D

    );

    /*

    A1not = A[1] nand A[1]

    A0not = A[0] nand A[0]

    D[0] = (A1not nand A0not) nand (A1not nand A0not)

    D[1] = (A1not nand A0) nand (A1not nand A0)

    D[2] = (A1 nand A0not) nand (A1 nand A0not)

    D[3] = (A1 nand A0) nand (A1 nand A0)

    */

    logic A1not, A0not , D0_not, D1_not, D2_not, D3_not;

    logic [3:0] buffer;

    NAND A1not_nand(A[1], A[1], A1not);

    NAND A0not_nand(A[0], A[0], A0not);

    NAND D0_not_nand(A1not, A0not, D0_not);

    NAND D1_not_nand(A1not, A[0], D1_not);

    NAND D2_not_nand(A[1], A0not, D2_not);

    NAND D3_not_nand(A[1], A[0], D3_not);

    NAND D0_nand(D0_not, D0_not, buffer[0]);

    NAND D1_nand(D1_not, D1_not, buffer[1]);

    NAND D2_nand(D2_not, D2_not, buffer[2]);

    NAND D3_nand(D3_not, D3_not, buffer[3]);

    assign D = E ? buffer : 4'b0000;

endmodule

module NAND(a, b, y);

input logic a, b;

output logic y;

    assign y = ~(a & b);

endmodule
```

6.2 TestBench Kodunun Yazılması ve Simülasyon

```
/* lab3_g5_p4.sv */

module lab3_g5_p3(

    //this is a 2x4 decoder

    input logic [1:0] A,

    input logic E,

    output logic [3:0] D

    );

    /*

    A1not = A[1] nand A[1]

    A0not = A[0] nand A[0]

    D[0] = (A1not nand A0not) nand (A1not nand A0not)

    D[1] = (A1not nand A0) nand (A1not nand A0)

    D[2] = (A1 nand A0not) nand (A1 nand A0not)

    D[3] = (A1 nand A0) nand (A1 nand A0)

    */

    logic A1not, A0not , D0_not, D1_not, D2_not, D3_not;

    logic [3:0] buffer;

    NAND A1not_nand(A[1], A[1], A1not);

    NAND A0not_nand(A[0], A[0], A0not);

    NAND D0_not_nand(A1not, A0not, D0_not);

    NAND D1_not_nand(A1not, A[0], D1_not);

    NAND D2_not_nand(A[1], A0not, D2_not);

    NAND D3_not_nand(A[1], A[0], D3_not);

    NAND D0_nand(D0_not, D0_not, buffer[0]);

    NAND D1_nand(D1_not, D1_not, buffer[1]);

    NAND D2_nand(D2_not, D2_not, buffer[2]);

    NAND D3_nand(D3_not, D3_not, buffer[3]);

    assign D = E ? buffer : 4'b0000;

endmodule

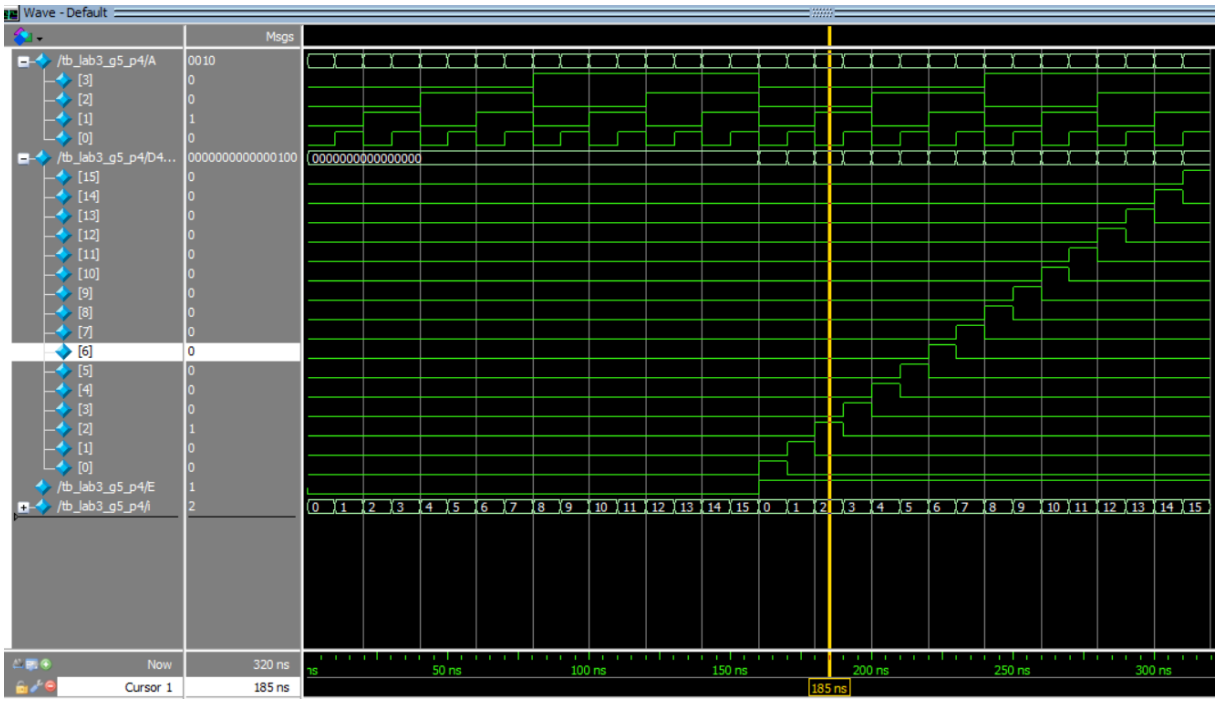
module NAND(a, b, y);

input logic a, b;

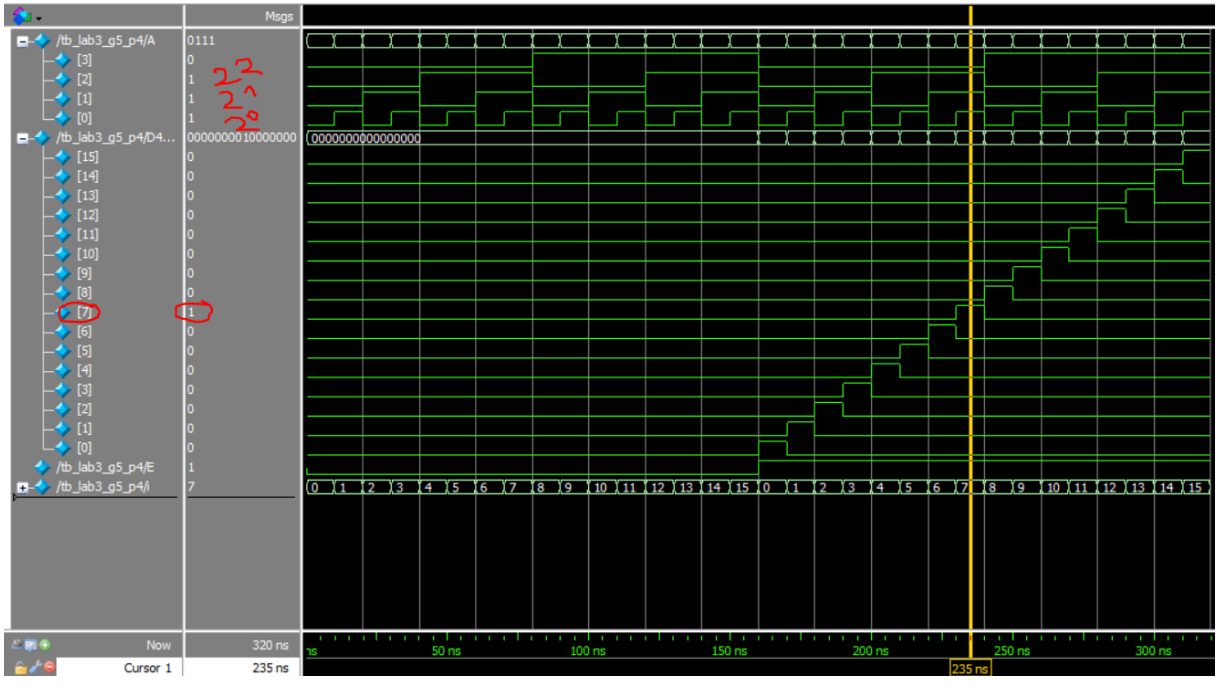
output logic y;

    assign y = ~(a & b);

endmodule
```

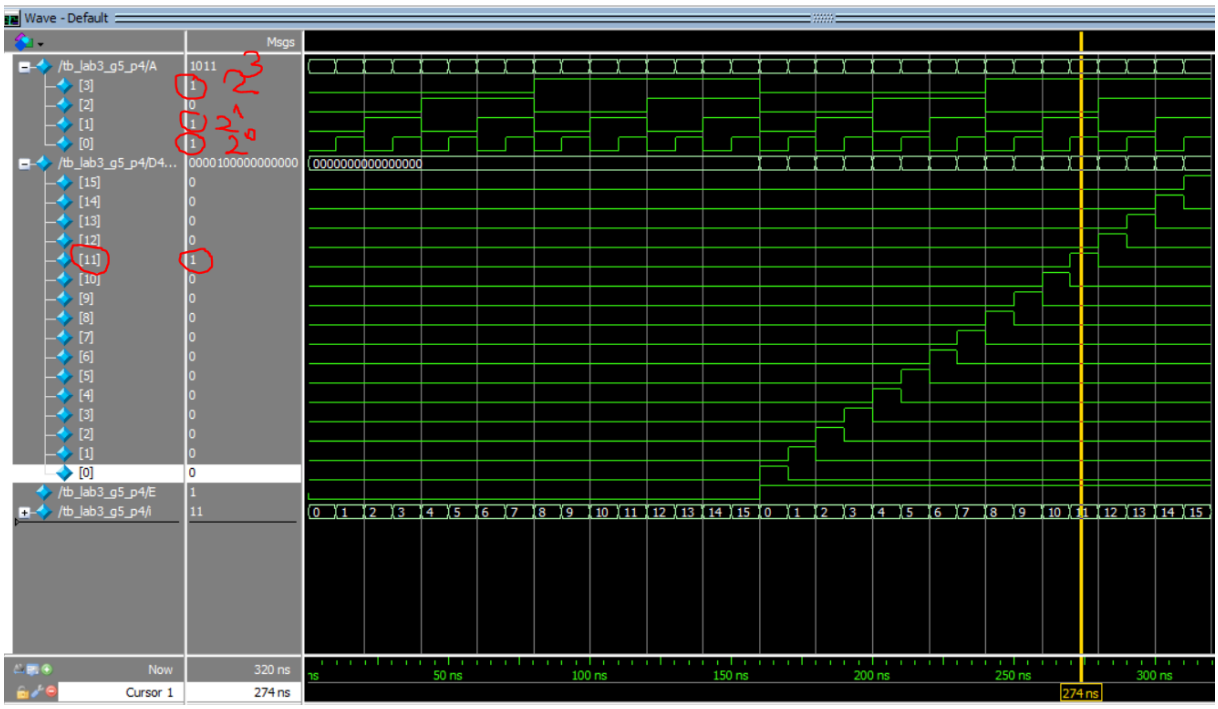


Şekil 16: Kodun Simülasyonu ve Waveform



Şekil 17

Yukarıda şekilde bir örnek gösterilmiştir. $2^2 + 2^1 + 2^0 = 7$ yaptığından 7.'nin 0,1 ve 2'yi taklit ettiği gözükür.



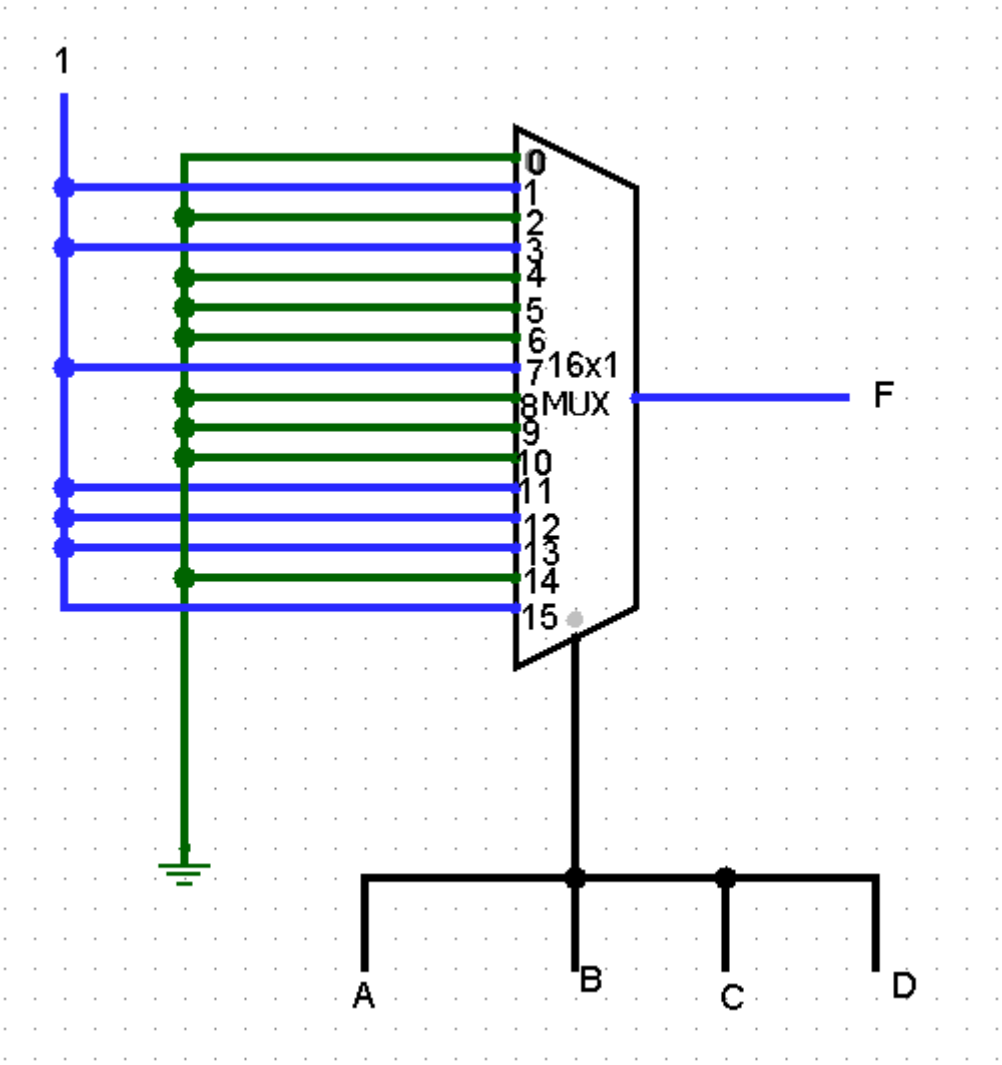
Şekil 18

Buradaki sinyalin kesitinde ise $2^3 + 2^1 + 2^0 = 11$ olduğundan 0,1 ve 3. ‘nün lojik ifadesini 11. taklit etmektedir.

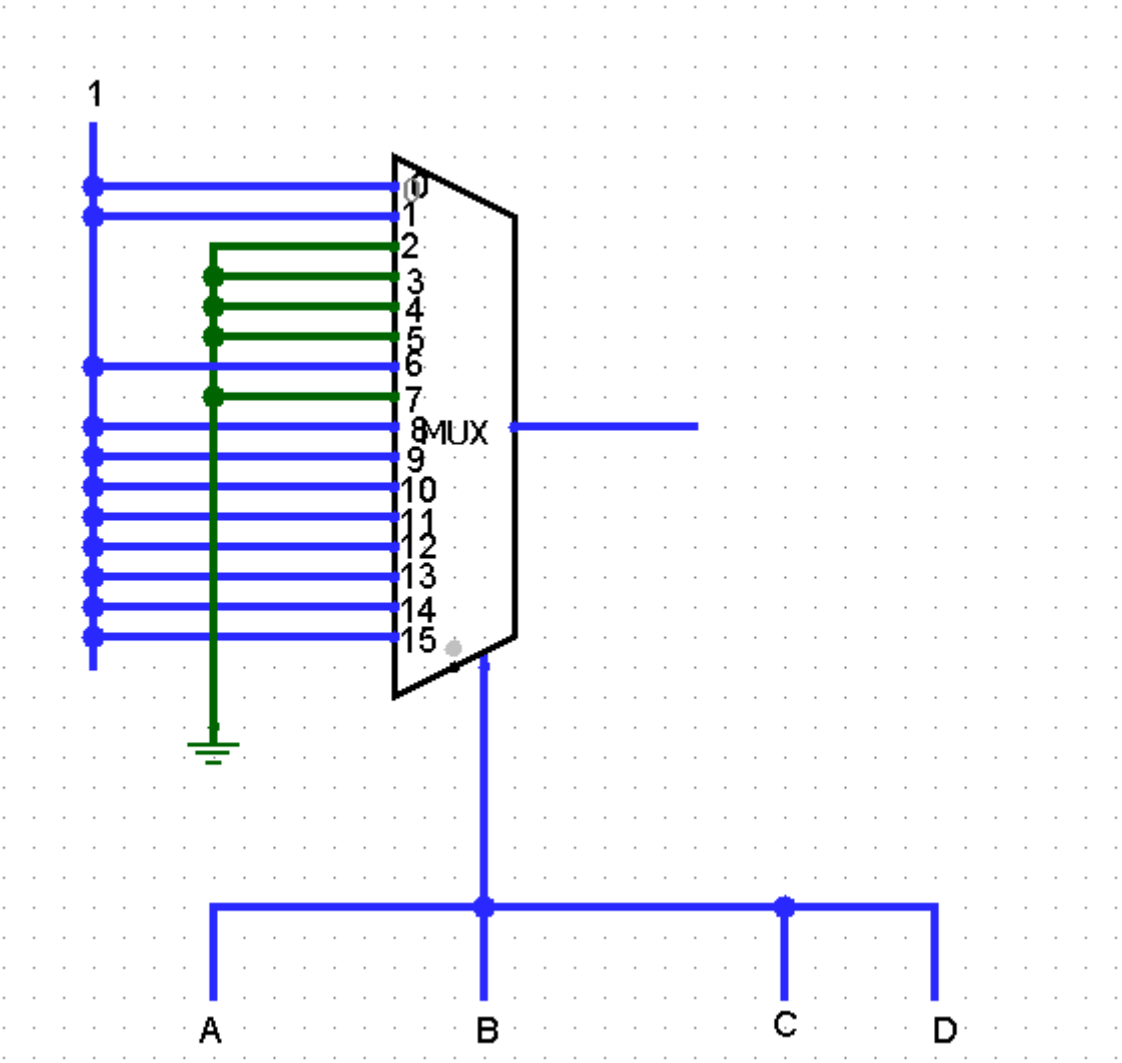
7 Problem 5 – Çoğullayıcı (MUX) ve Çözücüler (DECODER) ile Birleşik Devre Tasarımı

7.1 F1 ve F2 Fonksiyonlarının Donanım Tasarım Dilleri Kullanılarak Tasarımı

F1 fonksiyonu F1(A,B,C,D)= ∑{1,3,7,11,12,13,15} olarak ve F2 fonksiyonu F2(A,B,C,D)= ∏{2,3,4,5,7} olarak verilmiştir.



Şekil 19:F1 fonksiyonunun 16x1 mux ile tasarımı



Şekil 20:F2 fonksiyonunun 8x1 MUX ile tasarımı

A	B	C	D	F1 = MİN{1,3,7,11,12,13,15}	
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	0	2
0	0	1	1	1	3
0	1	0	0	0	4
0	1	0	1	0	5
0	1	1	0	0	6
0	1	1	1	1	7
1	0	0	0	0	8
1	0	0	1	0	9
1	0	1	0	0	10
1	0	1	1	1	11
1	1	0	0	1	12
1	1	0	1	1	13
1	1	1	0	0	14
1	1	1	1	1	15

Şekil 21:F1 Fonksiyonu İçin Truth Table

A	B	C	D	F1 = MİN{1,3,7,11,12,13,15}	
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	0	2
0	0	1	1	1	3
0	1	0	0	0	4
0	1	0	1	0	5
0	1	1	0	0	6
0	1	1	1	1	7
1	0	0	0	0	8
1	0	0	1	0	9
1	0	1	0	0	10
1	0	1	1	1	11
1	1	0	0	1	12
1	1	0	1	1	13
1	1	1	0	0	14
1	1	1	1	1	15

AB\CD		00	01	11	10
	00	0	1	1	0
	01	0	0	1	0
	11	1	1	1	0
	10	0	0	1	0

F1 = ABC' + A'B'D + CD

5 and, 2 or, 3 not kapısı gerekmektedir

Şekil 22: F1 Fonksiyonu İçin K-Map

A	B	C	D	F2 = MAX(2,3,4,5,7)	
0	0	0	0	1	0
0	0	0	1	1	1
0	0	1	0	0	2
0	0	1	1	0	3
0	1	0	0	0	4
0	1	0	1	0	5
0	1	1	0	1	6
0	1	1	1	0	7
1	0	0	0	1	8
1	0	0	1	1	9
1	0	1	0	1	10
1	0	1	1	1	11
1	1	0	0	1	12
1	1	0	1	1	13
1	1	1	0	1	14
1	1	1	1	1	15

Şekil 23:F2 Fonksiyonu için Truth Table

A	B	C	D	F2 = MAX(2,3,4,5,7)	
0	0	0	0	1	0
0	0	0	1	1	1
0	0	1	0	0	2
0	0	1	1	0	3
0	1	0	0	0	4
0	1	0	1	0	5
0	1	1	0	1	6
0	1	1	1	0	7
1	0	0	0	1	8
1	0	0	1	1	9
1	0	1	0	1	10
1	0	1	1	1	11
1	1	0	0	1	12
1	1	0	1	1	13
1	1	1	0	1	14
1	1	1	1	1	15

AB\CD	00	01	11	10
00	1	1	0	0
01	0	0	0	1
11	1	1	1	1
10	1	1	1	1

F2 = B'C'+A+BCD'

3 and ,2or, 3not kapısı kullanılmıştır

Şekil 24: F2 Fonksiyonu İçin K-Map

F1 ve F2 fonksiyonları için modül kodları aşağıdaki gibidir:

```
/* lab3_g5_p5.sv */

`timescale 1ns/1ns

module lab3_g5_p5(

    input logic A,B,C,D,

    output logic F1,F2

    );

    //F1 is the sum of minterms 1,3,7,11,12,13,15

    MUX16x1 mux1(16'b1011_1000_1000_1010, {A,B,C,D}, F1);

    MUX16x1 mux2(16'b1111_1111_0100_0011, {A,B,C,D}, F2);



endmodule



module MUX16x1(input logic [15:0] A , input logic [3:0] S, output logic Y);

    //A[0] = A; A[1] = A; A[2] = A; A[3] = A; A[4] = A; A[5] = A; A[6] = A; A[7] = A; A[8] = A; A[9] = A;
A[10] = A; A[11] = A; A[12] = A; A[13] = A; A[14] = A; A[15] = A; S = S;

    logic [15:0] Aarray;

    logic y1, y2;

    MUX8x1 mux1(A[7:0], S[2:0], y2);

    MUX8x1 mux2(A[15:8], S[2:0], y1);

    lab3_g5_p1 mux3({y1,y2}, S[3], Y);

endmodule

module MUX8x1(input logic [7:0] A , input logic [2:0] S, output logic Y);

    //A[0] = A; A[1] = A; A[2] = A; A[3] = A; A[4] = A; A[5] = A; A[6] = A; A[7] = A; S = S;

    logic [7:0] Aarray;

    logic y1, y2;

    MUX4x1 mux1(A[3:0], S[1:0], y2);

    MUX4x1 mux2(A[7:4], S[1:0], y1);

    lab3_g5_p1 mux3({y1,y2}, S[2], Y);

endmodule
```

```
module MUX4x1(input logic [3:0] A, input logic [1:0] S, output logic Y);
    //A[0] = A; A[1] = A; A[2] = A; A[3] = A; S = S;
    logic [3:0] Aarray;
    logic y1, y2;
    lab3_g5_p1 mux1(A[1:0], S[0], y2);
    lab3_g5_p1 mux2(A[3:2], S[0], y1);
    lab3_g5_p1 mux3({y1,y2}, S[1], Y);
endmodule

module lab3_g5_p1(
    //this is a 2x1 MUX
    input logic [1:0] A,
    input  logic S,
    output logic Y
    );
    // Y = ((S nand 1) nand A[0]) nand (S nand A[1])

    logic nand1, nand2, nand3;
    NAND n1(S, 1'b1, nand1);
    NAND n2(nand1, A[0], nand2);
    NAND n3(S, A[1], nand3);
    NAND n4(nand2, nand3, Y);
    //assign nand1 = ~(S & 1'b1);
    //assign nand2 = ~(nand1 & A[0]);
    //assign nand3 = ~(S & A[1]);
    //assign Y = ~(nand2 & nand3);

endmodule

module NAND(a, b, y);

    input logic a, b;
    output logic y;

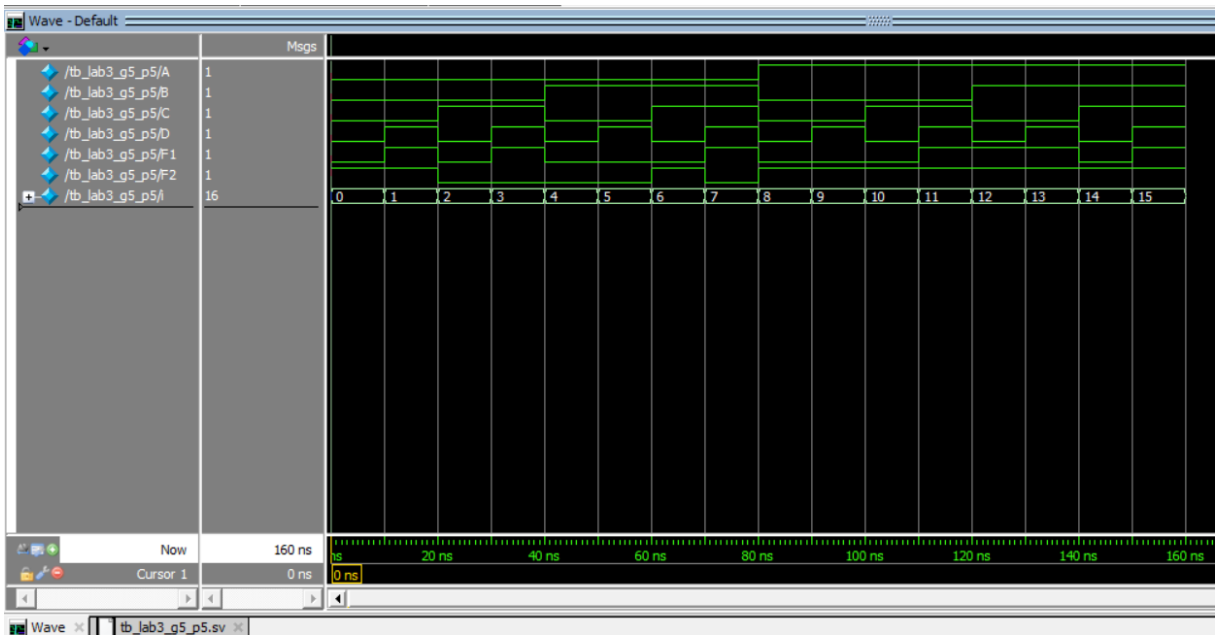
    assign y = ~(a & b);
endmodule
```

7.2 TestBench Kodu ve Simülasyon

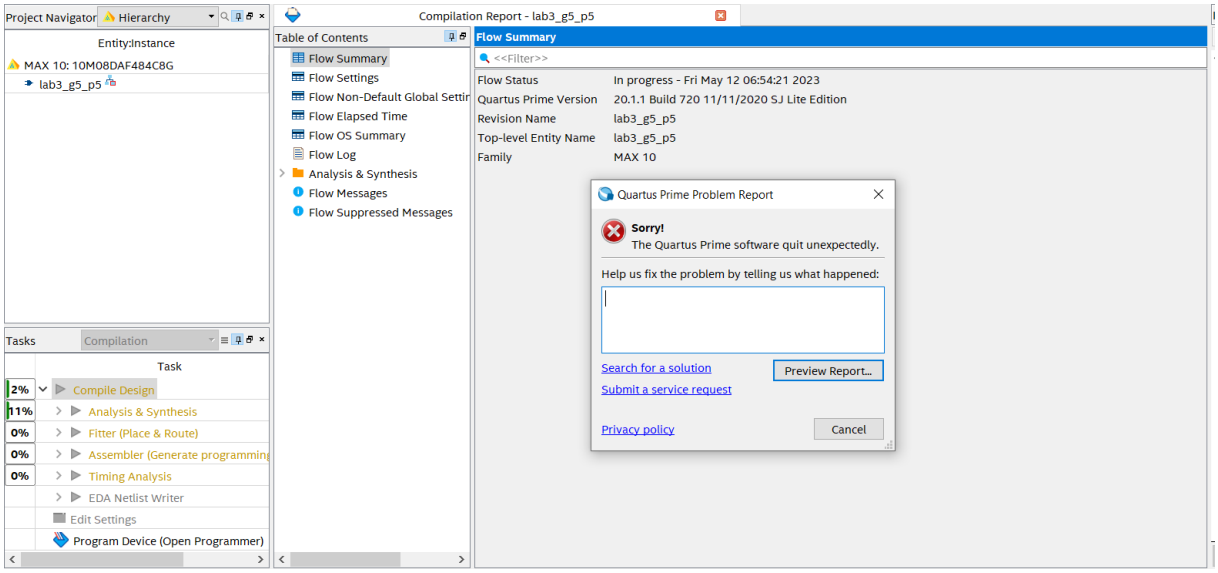
TestBench kodu şu şekildedir:

```
/* tb_lab3_g5_p5.sv */
`timescale 1ns/1ns
module tb_lab3_g5_p5();
    //logic [3:0] Anarray;
    logic A,B,C,D;
    logic F1, F2;

    //decoder3x8 dut0(A,E,D3x8);//device under test 0
    lab3_g5_p5 dut1(A,B,C,D, F1,F2);//device under test 1 //decoder 4x16
    integer i;
    initial begin
        for (i = 0; i < 16; i = i + 1) begin
            //A = Anarray;
            {A,B,C,D} = i;
            #10;//burada 10ns bekliyoruz
        end
        $stop;
    end
endmodule
```



Şekil 25: Kodun Simülasyonu ve Waveform



Şekil 26

Kodun Quartus Programı ile sentezlenmeye çalışıldığında internetteki tüm çözümler denendiği ve saatlerce uğraşıldığı halde çözüm bulunamamış ve Quartus programı hata vermiştir. Compile edilmeye çalışıldığında Quartus programı resimde görülen bu uyarıyı vermiştir. Çözüm için hata mesajı kopyalanıp internette araştırılmış ve önerilen çözümler denenmiş, Quartus programı silinip baştan yüklenmiş, kod sadeleştirilmiş ama hala hata alınmıştır. Dikkate almanızı arz ederiz.

8 Sonuç ve Yorumlar

Bu deney sonucunda MUX ‘ın bize sağladığı kolaylıkla, minterm cinsinden fonksiyonların MUX ile çok kolay bir şekilde gerçekleşebileceği öğrenildi. Aynı zamanda Decoder’ın yapısı hakkında bilgi elde edildi. Hiyerarşik tasarımı ile devrelerin daha kolay tasarlanması için nasıl kullanılabileceği öğrenildi. MUX kullanmanın veya kapı kullanmanın hangi durumlarda daha kolay veya ucuz olduğu incelendi. Devreler sentezlendi ve yapılar incelendi.